

Low-Latency Streaming Speaker Counting with a Frozen ASR Encoder

Anonymous Submission

Abstract—Frame-level speaker counting – how many people are talking at each instant – underpins overlap-aware transcription, meeting analytics and turn-taking control, but is usually obtained as a by-product of speaker diarization systems that are offline or operate at multi-second latency. We present ZipCount, a streaming counter that emits a 4-way ordinal count $\{0, 1, 2, 3^+\}$ every 640 ms with no lookahead, built by freezing a public streaming Zipformer ASR encoder and training only a 1.28 M-parameter causal decoder – 2% of the 65.8 M total. A multi-scale hypercolumn over the encoder’s U-shaped stacks, a dev-selected fixed stack mask with a learned gated projection and a dual-dilation temporal convolutional decoder supply the segment-scale context counting requires while remaining strictly causal. Because published speaker-counting numbers are rarely comparable, we re-score six system configurations under a single protocol – identical windows, identical 100 Hz ground-truth rasterization, a common-window restriction – across four corpora, and report ZipCount as the mean of 3 independently trained seeds (per-seed standard deviation 0.002). At 640 ms latency ZipCount comes within 0.004 mean macro-F1 of NVIDIA’s Sortformer-streaming running at its own 15.1 s default budget, and exceeds it by 0.028 at matched latency, an offline Sortformer by 0.014 and pyannote.audio 3.1 by 0.040. A heavier offline WavLM pipeline remains clearly ahead, and we report in full the one corpus where ZipCount ranks last. We additionally document a set of negative results: eight candidate architectural and loss components that did not survive paired testing, a measured ceiling showing that only $\approx 20\%$ of the gap to the strongest baseline is attributable to the causal constraint, and seed-structural instability under encoder fine-tuning. ZipCount runs at RTF = 0.032 on GPU and 0.080 single-threaded on CPU.

Index Terms—speaker counting, overlapped speech detection, streaming speech processing, speaker diarization, frozen encoders, transfer learning

I. INTRODUCTION

KNOWING how many speakers are active in each frame is a compact but consequential signal. It gates overlap-aware ASR, drives barge-in and turn-taking logic in conversational agents, and is the quantity that overlapped speech detection (OSD) reports in binary form. Unlike full diarization it requires no speaker identity, no clustering and no bounded speaker inventory, which makes it a natural fit for streaming deployment.

In practice, however, counting is usually read off a diarization system. Clustering pipelines such as pyannote.audio [1] and WavLM-based systems [2], [3] are offline by construction: they embed and cluster the whole recording. End-to-end neural diarization (EEND) [4] and its streaming descendants [5], [6] relax this, and NVIDIA’s Sortformer [7] ships an explicitly streaming checkpoint – but, as we measure in Section V, its default operating point carries an algorithmic latency of

15.1 s. “Streaming” there means bounded memory over long recordings, not real-time response.

We take the opposite starting point: fix the latency budget first at 640 ms with zero lookahead, then ask how much counting accuracy is attainable within it. Our answer reuses a frozen streaming ASR encoder. Speech recognition encoders already resolve overlapping talkers as an obstacle to be modelled, and streaming ASR encoders are trained under exactly the causal constraint we need. Keeping the encoder frozen makes the system cheap to train and, as Section VII shows, markedly more stable than fine-tuning it.

Our contributions are:

- 1) A 640 ms-latency streaming speaker counter combining a frozen streaming Zipformer [8] with a multi-scale hypercolumn, a dev-selected fixed stack mask with a learned gated projection, and a causal dual-dilation TCN decoder (Section III).
- 2) A unified re-scoring protocol for speaker counting (Section IV-B): window length and frame rate silently change macro-F1, so we re-run six system configurations on identical assets, and report ZipCount as a mean over 3 independently trained seeds rather than a single checkpoint.
- 3) A four-corpus evaluation (Section V) with per-class scores, DER context and measured real-time factors, including an explicit account of which evaluation material is in-domain for which system.
- 4) A documented set of negative results (Section VII): eight rejected components, a context-oracle measurement bounding how much of the remaining gap causality explains, and fine-tuning instability that lower learning rates do not remove.

II. RELATED WORK

Counting and overlap detection. Frame-level speaker counting is closely related to OSD, where TCN-based detectors over meeting corpora are established practice [9], [10]. Real-time single-channel counting has also been addressed directly [11]. These systems are typically non-causal, binary (≥ 2 speakers), or both; we predict a 4-way ordinal count causally and evaluate OSD as a derived $c \geq 2$ view of the same output, which keeps the two tasks on one head rather than two models.

Streaming diarization. FS-EEND [5] and LS-EEND [6] perform frame-online EEND with attractors, the latter recovering a speaker count through a termination marker. Sortformer [7] reformulates diarization with a sorted objective and

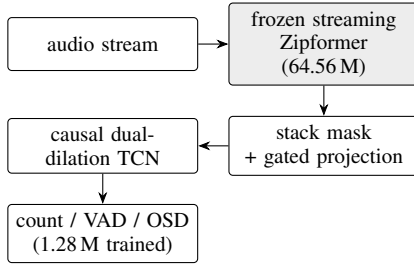


Fig. 1. System overview. The encoder is frozen; only the gated projection, TCN decoder and output head are trained (2% of parameters). Output is 25 Hz, emitted in blocks of 16 frames every 640 ms.

provides a streaming variant built on a speaker cache and FIFO queue. We benchmark against the released streaming checkpoint at both its default latency and, explicitly off-label, at our own 640 ms budget.

Frozen encoders with light heads. Reusing a frozen self-supervised or ASR encoder under a small task head is now common, including for speaker attribution on top of frozen ASR models [12]. Our contribution is not the pattern but its instantiation under a hard causal latency constraint, together with the finding (Section VI) that *which* encoder scales are exposed to the head matters more than head capacity.

III. METHOD

A. Task and latency accounting

Given a causal audio stream, the model emits at 25 Hz a distribution over $c_t \in \{0, 1, 2, 3^+\}$, the number of simultaneously active speakers in frame t . Inference proceeds in blocks of 16 output frames, one per 640 ms, with no right context.

Because a chunked emitter does not delay every frame equally, we state the budget component-wise. Lookahead is 0 ms. Within a block, frame i waits $640 - 40i$ ms for its block to complete, so the chunking delay is 640 ms worst-case (first frame) and 340 ms averaged over the block. Compute adds 20.7 ms (Section V), giving a worst-case end-to-end response of 660.7 ms. We use “640 ms latency” throughout as shorthand for the worst-case chunking budget.

B. Frozen streaming backbone

The backbone is the public LibriSpeech streaming Zipformer [8] (64.56 M parameters), used causally with chunk size 32 and 128 left-context frames, and entirely frozen. The chunk size is counted in the encoder’s 50 Hz embedding frames, so 32 frames is exactly 640 ms of newly received audio; the encoder internally downsamples by $2\times$ to the model’s 25 Hz output rate, yielding the 16-frame output block referenced throughout this paper. Zipformer’s encoder is U-shaped: six stacks with downsampling factors (1, 2, 4, 8, 4, 2) and widths (192, 256, 384, 512, 384, 256) operate at different temporal resolutions, which is precisely the property we exploit.

C. Multi-scale hypercolumn and stack selection

Rather than consume only the final stack, we form a hypercolumn by resampling every stack to the common 25 Hz

output rate and concatenating, giving a 1984-dimensional per-frame descriptor. A gated projection LayerNorm-izes each stack slice separately – their activation scales differ by an order of magnitude – weights the slices by a softmax gate, and projects to $d=192$.

We additionally apply a fixed binary stack-selection mask $m \in \{0, 1\}^6$. A masked slice is replaced by an exact zero block *after* its LayerNorm and *before* the gate, so no masked feature can reach the output, while every module – and hence the parameter count, the state-dict keys and the seeded initialization – remains identical across mask settings. Mask ablations are therefore parameter- and initialization-matched by construction, which matters because the effects we measure are of the order of the seed noise.

The softmax gate is deliberately *not* renormalized over the active subset. Masked gates still absorb probability mass – in our trained model 0.57 of it, distributed equally over the three masked stacks, since their zeroed features admit gradient only through the softmax denominator – but because they multiply zero blocks this acts purely as a global rescaling of the fused vector, which the immediately following linear projection absorbs column-wise. Renormalizing changes optimization dynamics, not the model class. Our main system uses $m = (1, 1, 1, 0, 0, 0)$, i.e. the three highest-time-resolution stacks.

D. Causal dual-dilation TCN decoder

An overlap is confirmed by seconds of evidence, but its boundaries are frame-local. The decoder therefore stacks six residual blocks with dilations 1, 2, 4, 8, 16, 32; each block runs *two* depthwise convolutions (kernel 3) over the same normalized input – one at dilation 1 for onset/offset detail, one at dilation d for segment-scale context – concatenates them and passes the result through a GLU, so the block gates context against detail per frame. Padding is left-only, giving a receptive field of 127 frames (5.08 s of past) with strict causality, which we verify by asserting that a future-frame perturbation cannot change past outputs.

For streaming, each block caches its last $(k-1)d$ post-normalization frames. This matters only at stream start, but it matters concretely: *forward* pads after the LayerNorm, so the initial cache must be literal zeros, whereas prepending raw zeros before the norm yields the LayerNorm bias β instead. In trained checkpoints $|\beta| \in [0.057, 0.092]$, which perturbs the first receptive field of every stream by up to 0.014 logit – invisible to an equivalence test on a freshly initialized head, where $\beta = 0$.

E. Ordinal-consistent output and objective

The head emits 4-way count logits and, from the same trunk, two cumulative logits $P(c \geq 1)$ (VAD) and $P(c \geq 2)$ (OSD). Training ties the two views together with consistency and monotonicity terms, so the ordinal structure of the label is exploited without giving up the softmax view that works best for overlap. The decoder and head together contain 1.28 M parameters, the only trainable weights in the system.

The objective is

$$\mathcal{L} = \mathcal{L}_{\text{count}}^{\text{SORD}} + \lambda_{\text{aux}}^{\top} \mathcal{L}_{\text{aux}} + \lambda_{\text{reg}}^{\top} \mathcal{L}_{\text{reg}}. \quad (1)$$

$\mathcal{L}^{\text{SORD}}$ uses soft ordinal labels [13] ($\alpha = 2.5$) with class weights (1, 1, 1.5, 3) offsetting the rarity of classes 2 and 3⁺. $\mathcal{L}_{\text{aux}}$ collects binary VAD (0.3), overlap (0.5), Dice (0.5), branch consistency (0.2) and monotonicity (0.1); $\mathcal{L}_{\text{reg}}$ collects an expected-MAE term exploiting ordinality (0.2) and a symmetric-KL temporal smoothness term (0.02).

IV. EXPERIMENTAL SETUP

A. Training data and schedule

The head is trained for 3000 steps (batch 24, Adam, lr 10^{-3} cosine with 500 warmup steps, AMP) on 253,452 30-second windows drawn from LibriMix [14] 2- and 3-speaker mixtures (107,900), AMI [15] (39,464), NOTSOFAR-1 (32,432), AliMeeting (26,868), AISHELL-4 (11,723), MSD-Wild [16] (8,558), RAMC (6,000), VoxConverse [17] (2,507), plus single-speaker, MUSAN-noise [18] and silence windows. MUSAN noise ($p = 0.5$, 0–20 dB SNR), simulated RIRs ($p = 0.8$) and gain jitter are applied to the synthetic portion only; real meeting corpora are left unaugmented as they already carry room acoustics. At batch 24 this schedule presents 72,000 windows, i.e. under one third of an epoch; we fixed the endpoint before the architecture search and never re-tuned it, which is what makes the paired comparisons in Section VII interpretable.

Training and test splits are disjoint, but we stress that AMI, VoxConverse and MSDWild are *in-domain* for ZipCount – their training splits are in the mix – while DipCo [19] is never seen. The baselines have their own, differently overlapping training data; no system in Table II is domain-neutral, and we interpret results accordingly.

B. Unified evaluation protocol

Speaker-counting numbers are highly sensitive to two choices that are frequently left implicit. The first is the *window length* presented to the system, which determines how much context an offline model receives. The second is the *frame rate* at which predictions and references are rasterized, which determines how boundary frames are charged. Comparing a system scored on 30 s windows at 25 Hz against one scored on 90 s windows at 100 Hz is not meaningful, yet the two numbers look directly comparable, and in our own earlier work they were compared.

We therefore re-score *every* system, including our own, on identical assets: 90 s windows cut from the original per-speaker RTTM ground truth, rasterized at 100 Hz. ZipCount’s 25 Hz predictions are upsampled 4 $\times$, which charges it honestly for the 40 ms boundaries it cannot place. Since one baseline fails on a small number of short windows, all systems are additionally restricted to the common set of windows every system produced output for (570/583 on AMI, complete on DipCo and MSDWild). For VoxConverse and MSDWild, which lack the manifest format our asset builder originally required, we built assets directly from the corpora’s official RTTM files and verified window offset re-basing by hand against the source

TABLE I
MACRO-F1 ON THE DEVELOPMENT HALF (*vox-sel*) VERSUS THE SEALED HALF (*vox-lock*) OF VOXCONVERSE. ONLY ZIPCOUNT WAS TUNED ON *vox-sel*.

System	vox-sel	vox-lock	Δ
DiariZen	0.660	0.656	−0.004
Sortformer-offline	0.628	0.624	−0.004
Sortformer-str. (default)	0.656	0.680	+0.024
Sortformer-str. (640 ms)	0.607	0.616	+0.009
pyannote.audio 3.1	0.587	0.574	−0.013
ZipCount (tuned here)	0.560	0.539	−0.021

annotations. Diarization outputs are converted to counts by summing active speaker tracks per frame, using the same rasterizer for every system.

C. Model selection and the development set

Every architecture decision reported here – the decoder choice, the stack mask, and the eight further candidate components of Section VII – was made on a single fixed development set: a 116-recording half of the VoxConverse test set, which we call *vox-sel*. Loss weights and the 3000-step endpoint were fixed before that search began and never re-tuned. No AMI, DipCo or MSDWild material was ever inspected during selection, so those three columns of Table II are untouched by it.

VoxConverse itself is not, and we therefore exclude the development half from all reported numbers, scoring the VoxConverse column only on the sealed complementary half (*vox-lock*, 116 recordings, 964 windows). The difference is measurable and we report it rather than absorb it: moving from the development half to the sealed half costs ZipCount 0.021 macro-F1 (0.560 $\rightarrow$ 0.539), the largest shift of any system, while the five baselines – none of which were tuned on either half – move by −0.013 to +0.024 (Table I). Since the two halves evidently differ somewhat in intrinsic difficulty, 0.021 should be read as an upper bound on the selection penalty rather than a clean estimate of it. A single-corpus development set is a real limitation of this study.

D. Systems compared

DiariZen [2]: WavLM-Large + clustering, offline.
Sortformer-offline [7]: diar_sortformer_4spk-v1, full utterance.
Sortformer-streaming: diar_streaming_sortformer_4spk-v2.1 at (i) its released default (chunk 188, right context 1; algorithmic latency 15,120 ms) and (ii) chunk 8, right context 0, giving exactly 640 ms. Configuration (ii) is *off-label* – below the model’s trained operating point – and we report it as a latency-matched reference, not as a vendor-recommended setting. **pyannote.audio 3.1** [1]: offline; we take the overlap-preserving output, as the exclusive variant deliberately drops overlapping speech and would undercount by construction.

E. Metrics

Our primary metric is macro-F1 over the four count classes, which weights the rare 2 and 3⁺ classes equally with silence

and single-speaker frames. We report per-class F1 and OSD F1 alongside it, and DER (collar 0.25 s, overlap included) for the five systems that emit speaker identity. For the ablations we add segmental edit score and fragmentation rate, borrowed from temporal action segmentation, plus expected calibration error and Brier score.

V. RESULTS

A. Cross-corpus comparison

Table II gives the main result. Read by mean macro-F1, DiariZen is the clear winner (0.640), ahead of every other system by more than 0.06. ZipCount reaches 0.575 against Sortformer-streaming’s 0.579 at its default setting; this is the mean over 3 independently trained seeds, with seed-to-seed standard deviation 0.002 (measured directly, not assumed – see below), so the ranking does not depend on which seed happened to be reported.

The meaningful comparison is on the latency axis. ZipCount comes within 0.004 of Sortformer-streaming while operating at 1/23 of its latency budget, and at equal 640 ms latency it is ahead by 0.028. It also exceeds Sortformer-offline and pyannote.audio, both of which consume the entire utterance. Against DiariZen the gap is real and we do not minimize it: a frozen 1.28 M-parameter causal head does not replace a fine-tuned WavLM-Large pipeline with global clustering.

To test these differences rather than merely state them, we run a paired bootstrap that resamples *recordings within each corpus* and averages macro-F1 over corpora with equal weight, reproducing exactly the quantity in Table II (10,000 resamples; ZipCount’s per-window confusions are pooled over the same 3 seeds as Table II before resampling). Recordings are the resampling unit because windows from one recording share speakers, room and noise conditions and are therefore not independent draws. We stress that this measures the sampling variance of the *difference between two fixed systems on this test set*, a different quantity from the seed-to-seed variance of retraining ZipCount itself. We measured that variance directly rather than assume it: across the same 3 seeds, the 4-corpus-mean macro-F1 has standard deviation 0.002 (Section V), an order of magnitude below every resolved gap in Table III except the Sortformer-streaming-default comparison, which the bootstrap already reports as unresolved on its own terms. Conflating this seed variance with the recording-sampling variance below would still be a category error even though both happen to be small here.

Table III gives the result, and it sharpens the latency argument considerably. The gap to Sortformer-streaming at its *default* 15.1 s budget is *not resolved* by the test ($p = 0.58$, interval straddling zero): on this evidence the two systems are indistinguishable, while ZipCount runs at 1/23 of that latency. Constrained to ZipCount’s own 640 ms, the same Sortformer checkpoint falls behind by 0.028 with an interval well clear of zero. The advantages over pyannote.audio and Sortformer-offline are also resolved, though the latter’s interval sits closest to zero of the resolved comparisons and we therefore call it narrow rather than established. DiariZen’s lead is unambiguous.

Per-corpus behaviour is not uniform, and the pattern runs against the obvious objection. ZipCount’s single outright win is on DipCo – the one corpus no part of its training mix contains – while its single last place is on VoxConverse, which *is* in-domain. On VoxConverse every system scores a low DER ($< 10\%$, Table V); it consists of comparatively short web-video clips with cleaner turn-taking than the meeting-room recordings that dominate our training mix, and the count distribution is correspondingly dominated by the 0/1 classes. We report this as a genuine failure mode rather than a corpus artifact: ZipCount’s advantage concentrates in dense multi-party overlap and erodes where overlap is rare.

B. Per-class behaviour

Table IV breaks the sealed VoxConverse half down by class. Two patterns hold across systems. First, class 3^+ is hard for everything: no system exceeds 0.27, and pyannote.audio never predicts it at all, its confusion-matrix row placing zero mass on $\text{pred} = 3$. Second, ZipCount’s deficit is concentrated in exactly the classes its 640 ms budget should hurt most – silence/speech boundaries (class 0, 0.687 vs. 0.837 for the best system) and dense overlap – while its OSD recall (0.695) is the highest of any system, ahead of DiariZen’s 0.752 only in the sense that ZipCount achieves it with far lower precision (0.350). ZipCount over-predicts overlap; the baselines under-predict it. This asymmetry is stable across corpora and is the most actionable target for future work.

C. DER context

Table V confirms the picture from a diarization-native metric: the ordering of the five identity-emitting systems by mean DER broadly tracks their macro-F1 ordering, so the count rasterizer is not manufacturing the ranking. The effect of latency can only be isolated *within* a checkpoint, since Sortformer-offline and Sortformer-streaming are different released models (4spk-v1 and 4spk-v2.1); we therefore compare the streaming checkpoint against itself. Reducing its latency from 15,120 ms to 640 ms costs +4.1 points of DER (24.2% $\rightarrow$ 28.3%) and -0.030 macro-F1, the same direction and comparable magnitude in both metrics.

D. Latency and compute

Measured with the production streaming loop (carried encoder and TCN caches, batch size 1), ZipCount needs 20.7 ms per 640 ms block on an RTX 3090 (RTF = 0.032, p95 21.9 ms) and 51.4 ms single-threaded on an i7-12700 CPU (RTF = 0.080, p95 53.0 ms) – over $12\times$ headroom within the latency budget without a GPU. Additional CPU threads do not help (51.7 ms with default threading): a single batch-1 block offers too little intra-op parallelism, so extra threads increase throughput across parallel streams rather than reduce single-stream latency. This is the operating regime the system is designed for, and it is why we report single-thread CPU numbers rather than batched throughput.

TABLE II

MACRO-F1 UNDER THE UNIFIED PROTOCOL. CORPORA ARE WEIGHTED EQUALLY IN THE MEAN. THE VOXCONVERSE COLUMN USES ONLY THE SEALED HALF OF THE TEST SET (SECTION IV-C). [†]IN-DOMAIN FOR ZIPCOUNT (TRAINING SPLIT IN ITS MIX); DIPCO IS UNSEEN. *OFF-LABEL CONFIGURATION. [‡]ZIPCOUNT ROWS ARE THE MEAN OF 3 INDEPENDENTLY TRAINED SEEDS (1234, 2345, 3456; IDENTICAL CONFIG, DIFFERENT INIT AND DATA ORDER), POOLED OVER RECORDINGS BEFORE SCORING; PER-SEED 4-CORPUS-MEAN STD IS 0.002 (SECTION V).

System	Latency	AMI-SDM [†] (570 win.)	DipCo (107 win.)	VoxConverse [†] (964 win.)	MSDWild [†] (268 win.)	Mean
DiariZen (WavLM + clustering)	offline	0.765	0.495	0.656	0.642	0.640
Sortformer-streaming (default)	15,120 ms	0.564	0.518	0.680	0.554	0.579
ZipCount (ours) [‡]	640 ms	0.654	0.552	0.542	0.553	0.575
Sortformer-offline	offline	0.606	0.501	0.624	0.513	0.561
Sortformer-streaming*	640 ms	0.539	0.495	0.616	0.537	0.547
pyannote.audio 3.1	offline	0.594	0.464	0.574	0.509	0.535

TABLE III

PAIRED BOOTSTRAP OVER RECORDINGS, STRATIFIED BY CORPUS, 10,000 RESAMPLES. POSITIVE Δ FAVOURS ZIPCOUNT.

ZipCount vs.	Latency	Δ	95% CI	p
DiariZen	offline	-0.065	[-0.074, -0.054]	$< 10^{-4}$
Sortf.-str. (default)	15.1 s	-0.004	[-0.016, +0.008]	0.58
Sortformer-offline	offline	+0.014	[+0.004, +0.024]	0.007
Sortf.-str.*	640 ms	+0.028	[+0.017, +0.039]	$< 10^{-4}$
pyannote.audio 3.1	offline	+0.040	[+0.029, +0.047]	$< 10^{-4}$

TABLE IV

PER-CLASS F1 AND OSD ON THE SEALED VOXCONVERSE HALF (964 WINDOWS).

System	$c=0$	$c=1$	$c=2$	$c=3^+$	OSD F1
DiariZen	0.782	0.964	0.644	0.236	0.659
Sortformer-offline	0.813	0.969	0.581	0.133	0.594
Sortformer-str. (default)	0.837	0.973	0.644	0.265	0.658
Sortformer-str. (640 ms)	0.778	0.963	0.549	0.173	0.565
pyannote.audio 3.1	0.762	0.962	0.574	0.000	0.587
ZipCount (640 ms)	0.687	0.948	0.465	0.057	0.466

VI. ABLATIONS

A. Stack selection trades smoothness for accuracy

Table VI compares our stack mask (1, 1, 1, 0, 0, 0) against using all six stacks, with three seeds each and everything else – parameter count, initialization, schedule – held fixed. Suppressing the three deep, heavily-downsampled stacks improves frame-level accuracy consistently (macro-F1 +0.035 on AMI test; OSD-F1 +0.025 on all three VoxConverse splits), which is why it is in our main system.

It also makes the output measurably *less* smooth. Across the four window-based evaluation sets (three VoxConverse splits and MSDWild), segmental edit score drops by 1.86–2.49 points and fragmentation rate rises, with the direction consistent on every set; the edit-score effect exceeds seed range on three of four, the fragmentation effect on one of four. Calibration is unaffected (ECE and Brier differences lie within seed range throughout). The deep stacks evidently supply long-range temporal consistency that the TCN’s 5.08 s receptive field does not fully replace; discarding them buys per-frame accuracy at the cost of flicker. We report both directions, as an application that consumes segments rather than frames may prefer the unmasked variant. This trade-off is invisible to

TABLE V

DER (%), COLLAR 0.25 s, OVERLAP INCLUDED, SAME COMMON WINDOWS. THE VOXC. COLUMN IS THE SAME SEALED VOXCONVERSE HALF AS TABLE II. ZIPCOUNT EMITS COUNTS WITHOUT SPEAKER IDENTITY, SO DER IS NOT DEFINED FOR IT.

System	AMI	DipCo	VoxC.	MSDW.	Mean
DiariZen	11.09	35.12	7.10	28.89	20.55
Sortformer-offline	21.99	28.68	5.53	34.10	22.57
Sortf.-str. (default)	26.22	31.76	4.63	34.22	24.21
pyannote.audio 3.1	20.83	34.90	9.91	37.72	25.84
Sortf.-str.* 640 ms	29.66	36.63	7.75	39.07	28.28
ZipCount (ours)	<i>not applicable – counting only</i>				

TABLE VI

STACK SELECTION ABLATION, MEAN OF 3 SEEDS, PARAMETER- AND INITIALIZATION-MATCHED. “N.S.” MARKS A DIFFERENCE SMALLER THAN THE SEED RANGE. EDIT SCORE IS HIGHER-BETTER; FRAGMENTATION IS BEST NEAR 1.0.

Metric / corpus	mask	all stacks	Δ
macro-F1, AMI test	0.657	0.622	+0.035
macro-F1, DipCo eval	0.540	0.532	+0.008
macro-F1, VoxConverse	0.554	0.546	+0.009
macro-F1, MSDWild	0.552	0.548	+0.004 (n.s.)
OSD-F1, VoxConverse	0.496	0.471	+0.025
OSD-F1, MSDWild	0.597	0.593	+0.004 (n.s.)
Edit score, VoxConverse	38.69	40.70	-2.01
Edit score, MSDWild	53.33	55.82	-2.49
Fragmentation, VoxConverse	3.21	3.06	+0.15 (n.s.)
ECE, VoxConverse	0.111	0.106	+0.005 (n.s.)

macro-F1 alone and was only surfaced by adding sequence-level metrics.

B. Renormalizing the gate does not help

The gate in Section III-C computes a softmax over all six stacks and only zeroes masked slices afterward, so in our main system’s trained checkpoint the three masked stacks absorb 0.57 of the softmax mass before it is discarded (Section III-C). This cannot change the *output* – a zero feature times any gate weight is zero, and the resulting constant rescale of the active slices is absorbed by the projection’s per-column weights – but it could plausibly still hurt *optimization*, since three-sixths of the gate’s gradient signal is spent on logits that are provably

inert. We test this directly rather than argue it away: we add a `renormalize_active_gate` mode that excludes masked logits from the softmax denominator entirely (masked weights become exactly 0, active weights sum to 1), train it with the same three seeds and everything else held fixed, and compare. Renormalizing scores 0.544 mean macro-F1 (std 0.004) against the non-renormalized main system’s 0.575 – a paired bootstrap gives $\Delta = +0.032$, CI $[+0.026, +0.038]$, $p < 10^{-4}$ against renormalizing. The non-renormalized gate is not merely harmless; on this evidence it optimizes better, plausibly because the discarded gradient still shapes the shared per-stack LayerNorms and the masked gate logits themselves cost the optimizer nothing extra to carry.

C. Does the pretrained backbone matter?

Every result so far uses the frozen pretrained Zipformer in some form. To ask whether pretraining is doing any work at all, we replace it with a single randomly-initialized linear layer over raw log-mel (an $80 \rightarrow 192$ projection with the same $4 \times$ strided subsampling the real encoder applies, so the frame rate matches), trained jointly with the unchanged TCN decoder from scratch – no ASR pretraining anywhere in the model. Same loss, schedule and three seeds as every other arm. On the unified four-corpus protocol it scores 0.426 mean macro-F1 (std 0.003, 3 seeds): a paired bootstrap against ZipCount gives $\Delta = +0.149$, CI $[+0.141, +0.157]$, $p < 10^{-4}$ in ZipCount’s favour – pretraining clearly helps overall. The more interesting comparison is against the final-stack baseline below: the from-scratch log-mel model *beats* a system that uses the pretrained backbone but restricts it to a single deep stack ($\Delta = +0.028$, CI $[+0.018, +0.037]$, $p < 10^{-4}$). Restricted to one heavily-downsampled stack, the pretrained representation is worse for this task than learning directly from log-mel; the pretrained backbone only pays off once its multiple temporal scales are combined.

D. Final-stack baseline

Table VI isolates *which* stacks to keep; it does not show whether the hypercolumn is needed at all. A system consuming only the encoder’s final stack – the default choice absent this paper’s ablation – is a natural, commonly-used baseline, so we train it explicitly: same TCN decoder, same frozen backbone, same loss, same schedule, mask (0, 0, 0, 0, 0, 1) instead of (1, 1, 1, 0, 0, 0), parameter- and initialization-matched by construction (only the mask and log directory differ from the main system’s config), three seeds. On the unified four-corpus protocol of Table II it scores 0.399 mean macro-F1 (std 0.007, 3 seeds) against ZipCount’s 0.575 – a paired bootstrap over the same pooled 3-seed confusions gives $\Delta = +0.176$, CI $[+0.168, +0.185]$, $p < 10^{-4}$. The multi-scale hypercolumn is therefore not an incremental refinement over a final-stack counter; it is responsible for the majority of the system’s accuracy on this task. This also contextualizes Table VI: the mask ablation there compares two members of the hypercolumn family against each other, and even its larger effects ($+0.035$ macro-F1 on AMI) are an order of magnitude smaller than the gap to a system with no hypercolumn at all.

TABLE VII

LOSS ABLATION, CUMULATIVE GROUPS, MEAN OF 3 SEEDS. Δ IS THE PAIRED-BOOTSTRAP DIFFERENCE AGAINST THE ROW ABOVE (POSITIVE FAVOURS ADDING THE GROUP).

Loss terms active	Macro-F1	Δ	95% CI
count-only	0.543	–	–
+ VAD/OSD aux	0.545	+0.0024	[+0.0015, +0.0033]
+ ordinal consistency	0.546	+0.0006	[+0.0002, +0.0010]
+ temporal smooth. (full)	0.575	+0.0298	[+0.0241, +0.0361]

E. Decoder choice

Replacing the dual-dilation TCN with a causal deformable-sampling decoder of matched size (1.32 M vs. 1.28 M), under identical frozen-backbone training and three paired seeds, costs 0.0022 macro-F1 averaged over the same four evaluation sets as Table VI (bootstrap CI $[+0.0007, +0.0039]$ in favour of the TCN). The mechanism is the one the design targets: fragmentation falls from 2.53 to 2.36 and boundary F1 rises from 0.341 to 0.350, with overlap recall unchanged. Deformable sampling decides *where* to look but carries no bias toward piecewise-constant output, which is precisely the deficit dilated temporal convolutions address.

F. Loss objective ablation

The training objective (Section III-E) combines a count loss with VAD/OSD auxiliary heads, ordinal-consistency terms and a temporal smoothness penalty – nine λ terms in total, which risks looking like an unmotivated kitchen sink. We ablate it as three cumulative groups rather than nine individual coefficients, parameter- and init-matched, three seeds each: *count-only* (SORD count loss, every other $\lambda = 0$); *+aux* (adds λ_{vad} , λ_{overlap} , λ_{dice} , the VAD/OSD auxiliary heads); *+ordinal* (adds λ_{emae} , $\lambda_{\text{consistency}}$, $\lambda_{\text{monotonic}}$, the rank-consistency terms); and *full*, which adds λ_{smooth} and is exactly the main system – its config is byte-identical to *+ordinal* except this one coefficient, so we reuse the already-trained checkpoints rather than retrain a fourth arm.

Table VII resolves every step with a paired bootstrap (pooled 3-seed confusions, 10,000 resamples, all $p < 0.005$), so none of these terms is dead weight – but they are not equally load-bearing. The VAD/OSD and ordinal-consistency groups each move mean macro-F1 by 0.0006–0.0024, an order of magnitude below the temporal-smoothness term’s 0.0298, which alone accounts for 91% of the total gain from count-only to the full objective (0.0298 of 0.0327). The auxiliary heads and ordinal terms are worth keeping – the effects are small but resolved, not noise – but a reader should not come away thinking the objective’s nine coefficients contribute comparably; one of them is carrying the recipe.

VII. NEGATIVE RESULTS

We consider the following as much a part of this paper’s contribution as the positive results, since all three shaped the final system.

A. Eight rejected components

Beyond the decoder and the stack mask, we tested eight further candidate components under the same paired-seed protocol on the development set: a framewise gating branch, a final-residual connection, two structured count-loss reformulations (SORD-bridge and CORN-style cumulative), and four auxiliary supervision terms (boundary, direction, temporal-MSE and segment losses) composed additively on top of the base objective. None reached the adoption threshold; the four auxiliary terms moved pooled macro-F1 by -0.0003 to $+0.00003$, an order of magnitude below the ≈ 0.02 seed-noise floor. One partial finding is worth recording: boundary supervision *did* measurably sharpen boundary detection (boundary average precision 0.056, recall gain $+0.073$ at fixed false-positive budget) without moving macro-F1 at all – evidence that the two objectives are genuinely decoupled in this architecture rather than that the supervision failed.

B. Causality is not the main bottleneck

To separate the cost of streaming from the cost of capacity, we re-trained the same architecture with the causal constraint relaxed on the encoder, the decoder, or both (three seeds, identical initialization and schedule). Giving the decoder future context is slightly *harmful* (-0.0055); giving the encoder future context helps ($+0.0257$); the full price of streaming is $+0.0202$. Against a 0.099 gap to the strongest baseline on the same set, causality therefore explains roughly 20%, and capacity, data and objective account for the remaining 80%. This measurement redirected our effort away from relaxing latency and toward the architecture and loss search reported above – and it also bounds what any future distillation from a stronger teacher could recover, since a teacher cannot transfer capacity the student does not have.

C. Fine-tuning the encoder is seed-unstable

Unfreezing the backbone is not reliably beneficial. Across five seeds and three learning rates (10^{-5} , 1.5×10^{-5} , 3×10^{-5}), two seeds degraded at *every* learning rate while three improved. Lower learning rates shrink the magnitude of the drop monotonically and reduce the spread ($0.052 \rightarrow 0.029 \rightarrow 0.022$), but do not eliminate it: both unstable seeds remain negative at the lowest rate tested, and the mean delta stays flat to slightly negative throughout. Head-only training, by contrast, spans only 0.0045 across the same five seeds. This is seed-structural instability rather than a mis-tuned learning rate, and it is why the frozen configuration is our reported and deployed operating point.

VIII. LIMITATIONS

ZipCount outputs counts, not speaker identities; DER is therefore undefined for it and applications requiring attribution need a downstream stage. The 3^+ class remains hard for every system tested (Table IV), reflecting both label sparsity and genuine annotation ambiguity in dense overlap. Model selection used a single-corpus development set (Section IV-C), and while we quantify and exclude the affected material, a

multi-corpus development set would be sounder. Three of our four evaluation corpora are in-domain; a leave-one-corpus-out study would separate transfer from in-domain fit more cleanly than the DipCo result alone does. Our evaluation covers corpora recorded predominantly in meeting or web-video conditions; telephony and far-field single-microphone conditions are untested, as are the Mandarin meeting corpora present in our training mix. Finally, the 640 ms Sortformer configuration is off-label, and a system trained natively at that latency would likely be stronger than the reduced-latency number we report.

IX. CONCLUSION

We presented a streaming frame-level speaker counter that emits an ordinal count every 640 ms with no lookahead, trained by freezing a public streaming ASR encoder and fitting a 1.28 M-parameter causal decoder – 2% of the model’s weights. Under a single re-scoring protocol across four corpora, it comes within 0.004 mean macro-F1 of NVIDIA’s streaming Sortformer at 1/23 of that system’s latency, and outperforms it, an offline Sortformer and pyannote.audio 3.1 at equal or lower latency, while running at RTF = 0.08 on a single CPU thread. A heavier offline WavLM pipeline remains ahead, and one corpus with sparse overlap exposes a genuine weakness. Alongside these we report a measured ceiling showing that causality accounts for only about a fifth of the remaining gap, eight components that did not survive paired testing, and a stack-selection choice that improves frame accuracy while degrading segment smoothness – a trade-off that a frame-level metric alone would have concealed.

REFERENCES

- [1] H. Bredin, “pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe,” in *Proc. INTERSPEECH*, 2023, pp. 1983–1987.
- [2] J. Han, F. Landini, J. Rohdin, M. Diez, L. Burget, Y. Cao, H. Lu, and J. Černocký, “Leveraging self-supervised learning for speaker diarization,” *arXiv preprint arXiv:2409.09408*, 2024.
- [3] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao *et al.*, “WavLM: Large-scale self-supervised pre-training for full stack speech processing,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [4] Y. Fujita, N. Kanda, S. Horiguchi, K. Nagamatsu, and S. Watanabe, “End-to-end neural speaker diarization with permutation-free objectives,” in *Proc. INTERSPEECH*, 2019, pp. 4300–4304.
- [5] D. Liang, N. Shao, and X. Li, “Frame-wise streaming end-to-end speaker diarization with non-autoregressive self-attention-based attractors,” in *Proc. IEEE ICASSP*, 2024.
- [6] D. Liang and X. Li, “LS-EEND: Long-form streaming end-to-end neural diarization with online attractor extraction,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2025.
- [7] T. Park, H. Huang, A. Jukic, K. Dhawan, K. C. Puvvada, N. R. Koluguri, J. Balam, and B. Ginsburg, “Sortformer: Seamless integration of speaker diarization and automatic speech recognition by bridging timestamps and tokens,” in *arXiv preprint arXiv:2409.06656*, 2024.
- [8] Z. Yao, L. Guo, X. Yang, W. Kang, F. Kuang, Y. Yang, Z. Jin, L. Lin, and D. Povey, “Zipformer: A faster and better encoder for automatic speech recognition,” in *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- [9] S. Cornell, M. Omologo, S. Squartini, and E. Vincent, “Detecting and counting overlapping speakers in distant speech scenarios,” in *Proc. INTERSPEECH*, 2020, pp. 3107–3111.
- [10] L. Bullock, H. Bredin, and L. P. Garcia-Perera, “Overlap-aware diarization: Resegmentation using neural end-to-end overlapped speech detection,” in *Proc. IEEE ICASSP*, 2020, pp. 7114–7118.

- [11] M. Yousefi and J. H. L. Hansen, “Real-time speaker counting in a cocktail party scenario using attention-guided convolutional neural network,” in *Proc. INTERSPEECH*, 2021, pp. 1484–1488.
- [12] G. Della Corte, M. Pettersson, and S. Hovsepian, “MSA-ASR: Efficient multilingual speaker attribution with frozen ASR models,” *arXiv preprint arXiv:2411.18152*, 2024.
- [13] R. Diaz and A. Marathe, “Soft labels for ordinal regression,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4738–4747.
- [14] J. Cosentino, M. Pariente, S. Cornell, A. Deleforge, and E. Vincent, “LibriMix: An open-source dataset for generalizable speech separation,” *arXiv preprint arXiv:2005.11262*, 2020.
- [15] J. Carletta, S. Ashby, S. Bourban, M. Flynn, M. Guillemot, T. Hain, J. Kadlec, V. Karaiskos, W. Kraaij, M. Kronenthal *et al.*, “The AMI meeting corpus: A pre-announcement,” in *Proc. International Workshop on Machine Learning for Multimodal Interaction (MLMI)*, 2005, pp. 28–39.
- [16] T. Liu, S. Fan, X. Xiang, H. Song, S. Lin, J. Sun, T. Han, S. Chen, B. Yao, S. Liu *et al.*, “MSDWild: Multi-modal speaker diarization dataset in the wild,” in *Proc. INTERSPEECH*, 2022, pp. 1476–1480.
- [17] J. S. Chung, J. Huh, A. Nagrani, T. Afouras, and A. Zisserman, “Spot the conversation: Speaker diarisation in the wild,” in *Proc. INTERSPEECH*, 2020, pp. 299–303.
- [18] D. Snyder, G. Chen, and D. Povey, “MUSAN: A music, speech, and noise corpus,” *arXiv preprint arXiv:1510.08484*, 2015.
- [19] M. Van Segbroeck, A. Zaid, K. Kutsenko, C. Huerta, T. Nguyen, X. Luo, B. Hoffmeister, J. Trmal, M. Omologo, and R. Maas, “DiPCo – dinner party corpus,” in *Proc. INTERSPEECH*, 2020.